BÁO CÁO BÀI TẬP LẬP TRÌNH MẠNG

Họ tên: Hoàng Tuấn Minh

MSSV: 20235376

Mã lớp: 168503

Link bài tập: hoangtminh/Lap-trinh-mang

1. Bài tập chat_server

- Sử dụng hàm poll(), viết chương trình chat_server thực hiện các chức năng sau: Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp: "client_id: client_name" trong đó client_name là tên của client, xâu ký tự viết liền.
- Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id "abc" gửi "xin chao" thì các client khác sẽ nhận được: "abc: xin chao" hoặc trước ví dụ: "2023/05/06 11:00:00PM abc: xin chao"

chat_server.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <poll.h>
#include <time.h>

#define MAX_CLIENTS 100

typedef struct {
    int fd;
    char name[50];
    int registered;
} ClientInfo;

int main() {
    int listener = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(8080);

    bind(listener, (struct sockaddr *)&addr, sizeof(addr));
    listen(listener, 5);

    ClientInfo clients[MAX_CLIENTS];
    struct pollfd fds[MAX_CLIENTS + 1];
    int nClients = 0;

    fds[0].fd = listener;
    fds[0].events = POLLIN;
```

Connect Git Graph Ln 92, Col 62 (1 selected) Spaces: 4 UTF-8 CRLF C Go Live

Code chat_server 1

```c
    while (1) {
        for (int i = 0; i < nClients; i++) {
            fds[i + 1].fd = clients[i].fd;
            fds[i + 1].events = POLLIN;
        }
        int poll_count = poll(fds, nClients + 1, -1);

        if (poll_count < 0) {
            perror("poll");
            break;
        }
        if (fds[0].revents & POLLIN) {
            int client_fd = accept(listener, NULL, NULL);
            if (nClients < MAX_CLIENTS) {
                clients[nClients].fd = client_fd;
                clients[nClients].registered = 0;
                nClients++;

                char *msg = "Hay nhap ten theo cu phap: client_id: client_name\n";
                send(client_fd, msg, strlen(msg), 0);
                printf("Client moi ket noi: FD %d\n", client_fd);
            } else {
                close(client_fd);
            }
        }
        for (int i = 0; i < nClients; i++) {
            if (fds[i + 1].revents & POLLIN) {
                char buf[256];
                int ret = recv(clients[i].fd, buf, sizeof(buf) - 1, 0);

                if (ret <= 0) {
                    printf("Client %s (FD %d) da thoat.\n", clients[i].name, clients[i].fd);
                    close(clients[i].fd);
                    clients[i] = clients[nClients - 1];
```

Connect Git Graph Ln 92, Col 62 (1 selected) Spaces: 4 UTF-8 CRLF C Go Live

Code chat_server 2

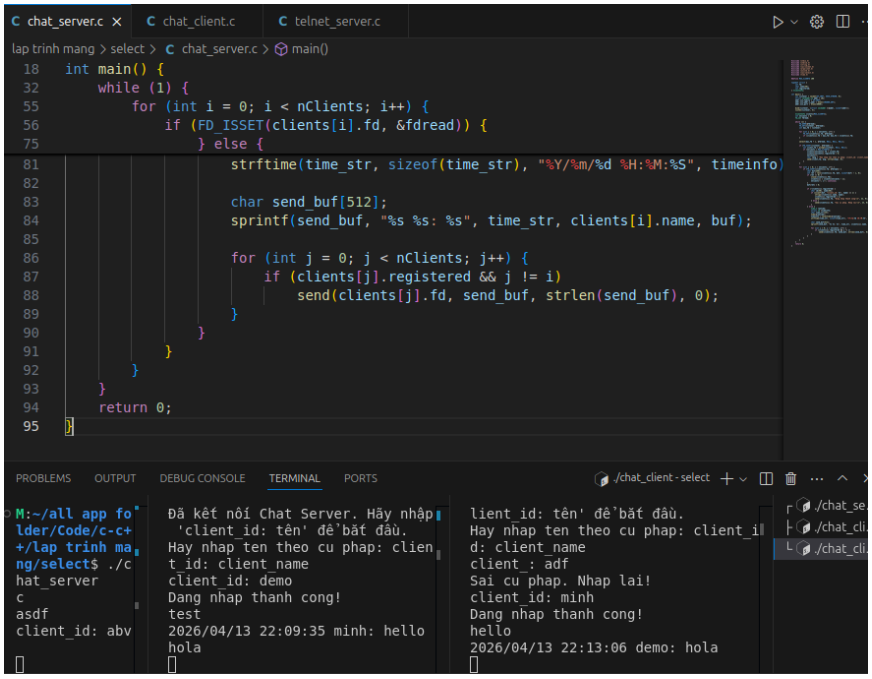
chat_server.c > main()

```c
                    nClients--;
                    i--;
                    continue;
                }

                buf[ret] = 0;
                if (!clients[i].registered) {
                    char name[50];
                    if (sscanf(buf, "client_id: %s", name) == 1) {
                        strcpy(clients[i].name, name);
                        clients[i].registered = 1;
                        send(clients[i].fd, "Dang nhap thanh cong!\n", 22, 0);
                    } else {
                        send(clients[i].fd, "Sai cu phap. Nhap lai!\n", 23, 0);
                    }
                } else {
                    time_t rawtime;
                    struct tm *timeinfo;
                    char time_str[25];
                    time(&rawtime);
                    timeinfo = localtime(&rawtime);
                    strftime(time_str, sizeof(time_str), "%Y/%m/%d %H:%M:%S", timeinfo);      Pin sel

                    char send_buf[512];
                    sprintf(send_buf, "%s %s: %s", time_str, clients[i].name, buf);

                    for (int j = 0; j < nClients; j++) {
                        if (clients[j].registered && j != i) {
                            send(clients[j].fd, send_buf, strlen(send_buf), 0);
                        }
                    }
                }
            }
        }
```

Connect Git Graph Ln 92, Col 62 (1 selected) Spaces: 4 UTF-8 CRLF C Go Live

Code chat_server 3

Kết quả:



Kết quả chat_server

2. Bài tập telnet_server

- Sử dụng hàm poll(), viết chương trình telnet_server thực hiện các chức năng sau: Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass
- Ví dụ: admin admin | guest nopass
- Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.
- Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client. Dùng hàm system(“dir > out.txt”) để thực hiện lệnh. dir là ví dụ lệnh dir mà client gửi.
- > out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

telnet_server.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <poll.h>

#define MAX_CLIENTS 100

typedef struct {
    int fd;
    int authenticated;
    char client_ip[INET_ADDRSTRLEN];
} TelnetClient;

int check_login(char *user, char *pass) {
    FILE *f = fopen("db.txt", "r");
    if (!f) {
        printf("Khong mo duoc file\n");
        return 0;
    }
    char u[50], p[50];
    while (fscanf(f, "%s %s", u, p) != EOF) {
        if (strcmp(user, u) == 0 && strcmp(pass, p) == 0) {
            fclose(f);
            return 1;
        }
    }
    fclose(f);
    return 0;
}

int main() {
```

Connect Git Graph Ln 10, Col 1 Spaces: 4 UTF-8 CRLF C Go Live

Code telnet_server 1

telnet_server.c > main()

```c
int main() {
    int listener = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(8082);

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind() failed");
        return 1;
    }
    listen(listener, 5);
    printf("Telnet Server dang chay tren port 8082...\n");

    TelnetClient clients[MAX_CLIENTS];
    struct pollfd fds[MAX_CLIENTS + 1];
    int nClients = 0;

    fds[0].fd = listener;
    fds[0].events = POLLIN;

    while (1) {
        for (int i = 0; i < nClients; i++) {
            fds[i + 1].fd = clients[i].fd;
            fds[i + 1].events = POLLIN;
        }

        int poll_count = poll(fds, nClients + 1, -1);
        if (poll_count < 0) {
            perror("poll() failed");
            break;
        }

        if (fds[0].revents & POLLIN) {
```

Connect Git Graph Ln 105, Col 1 Spaces: 4 UTF-8 CRLF C Go Live

Code telnet_server 2

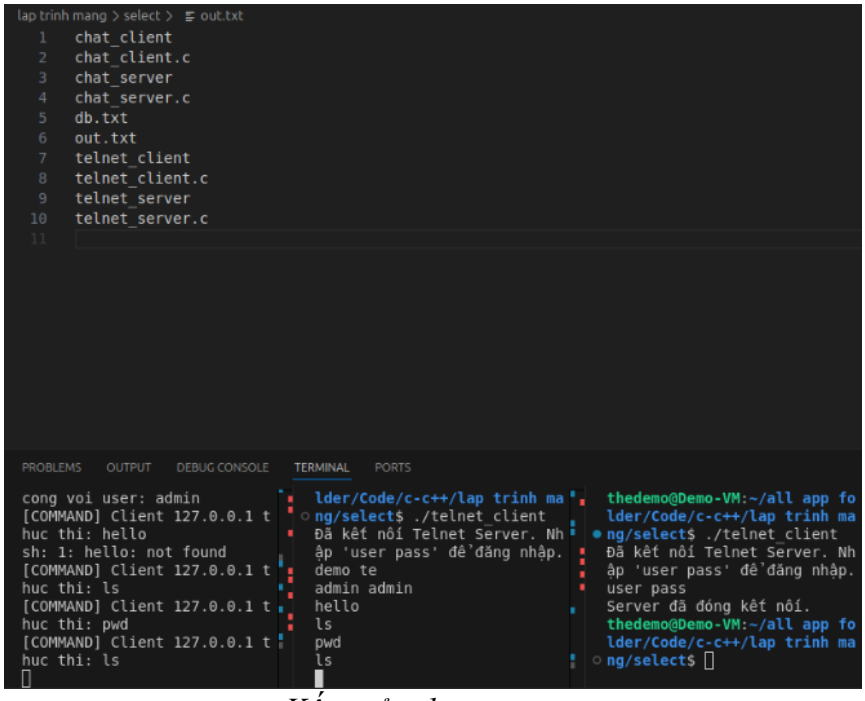
```c
telnet_server.c > main()
            struct sockaddr_in client_addr;
            socklen_t client_len = sizeof(client_addr);
            int client_fd = accept(listener, (struct sockaddr *)&client_addr, &client_len);

            if (nClients < MAX_CLIENTS) {
                clients[nClients].fd = client_fd;
                clients[nClients].authenticated = 0;
                inet_ntop(AF_INET, &client_addr.sin_addr, clients[nClients].client_ip, INET_ADDRSTRLEN);

                printf("Client moi tu IP: %s (FD: %d)\n", clients[nClients].client_ip, client_fd);

                nClients++;
                send(client_fd, "User Pass: ", 11, 0);
            } else {
                printf("Server day, tu choi ket noi tu FD: %d\n", client_fd);
                close(client_fd);
            }
        }

        for (int i = 0; i < nClients; i++) {
            if (fds[i + 1].revents & POLLIN) {
                char buf[256];
                int ret = recv(clients[i].fd, buf, sizeof(buf) - 1, 0);

                if (ret <= 0) {
                    printf("Client %s (FD: %d) da thoat\n", clients[i].client_ip, clients[i].fd);
                    close(clients[i].fd);
                    clients[i] = clients[nClients - 1];
                    nClients--;
                    continue;
                }

                buf[ret] = 0;
                while(ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) {
Connect  Git Graph                    Ln 105, Col 1  Spaces: 4  UTF-8  CRLF  C  Go Live  Win32
```

Code telnet_server 3

```c
telnet_server.c > main()
                if (!clients[i].authenticated) {
                    char user[50], pass[50];
                    if (sscanf(buf, "%s %s", user, pass) == 2 && check_login(user, pass)) {
                        clients[i].authenticated = 1;
                        printf("Client %s dang nhap thanh cong: %s\n", clients[i].client_ip, use
                        send(clients[i].fd, "Login success. Enter command: ", 30, 0);
                    } else {
                        printf("Client %s dang nhap that bai: %s\n", clients[i].client_ip, buf);
                        send(clients[i].fd, "Login failed. Try again: ", 25, 0);
                    }
                }
                // Nếu đã đăng nhập: Thực thi lệnh hệ thống
                else {
                    printf("[COMMAND] Client %s: %s\n", clients[i].client_ip, buf);
                    char cmd[512];
                    sprintf(cmd, "%s > out_%d.txt", buf, clients[i].fd);
                    system(cmd);

                    FILE *f = fopen(cmd + strlen(buf) + 3, "r");
                    if (f) {
                        char result[2048];
                        size_t bytes_read = fread(result, 1, sizeof(result) - 1, f);
                        result[bytes_read] = 0;
                        fclose(f);
                        if (bytes_read > 0) {
                            send(clients[i].fd, result, bytes_read, 0);
                        } else {
                            send(clients[i].fd, "Command executed with no output.\n", 33, 0);
                        }
                    }
                }
            }
        }
    }
    close(listener);
Connect  Git Graph                    Ln 103, Col 18  Spaces: 4  UTF-8  CRLF  C  Go Live
```

Code telnet_server 4

- **Kết quả:**



Kết quả telnet_server